



Overview

Concepts

Overview

Introduction

NautilusTrader is an open-source, production-grade, Rust-native engine for multi-asset, multi-venue trading systems.

The system spans research, deterministic simulation, and live execution within a single event-driven architecture, with Python serving as the control plane for strategy logic, configuration, and orchestration.

This separation provides the performance and safety of a compiled trading engine with the flexibility of Python for system composition and strategy development. Trading systems can also be written entirely in Rust for mission-critical workloads.

The same execution semantics and deterministic time model operate in both research and live systems. Strategies deploy from research to production with no code changes, providing research-to-live parity and reducing the divergence that typically introduces deployment risk.

NautilusTrader is asset-class-agnostic. Any venue with a REST API or WebSocket feed can be integrated through modular adapters. Current integrations span crypto exchanges (CEX and DEX), traditional markets (FX, equities, futures, options), and betting exchanges.

Features

- Fast: Rust core with asynchronous networking using [tokio](#).
- Reliable: Type- and thread-safety backed by Rust, with optional Redis-backed state persistence.

- **Portable:** Runs on Linux, macOS, and Windows. Deploy using Docker.
- **Flexible:** Modular adapters integrate any REST API or WebSocket feed.
- **Advanced:** Time in force `IOC`, `FOK`, `GTC`, `GTD`, `DAY`, `AT_THE_OPEN`, `AT_THE_CLOSE`, advanced order types and conditional triggers. Execution instructions `post-only`, `reduce-only`, and icebergs. Contingency orders including `OCO`, `OUO`, `OTO`.
- **Customizable:** User-defined components, or assemble entire systems from scratch using the cache and message bus.
- **Backtesting:** Multiple venues, instruments, and strategies simultaneously using historical quote tick, trade tick, bar, order book, and custom data with nanosecond resolution.
- **Live:** Identical strategy implementations between research and live deployment.
- **Multi-venue:** Run market-making and cross-venue strategies across multiple venues simultaneously.
- **AI training:** Engine fast enough to train AI trading agents (RL/ES).

Why NautilusTrader?

Trading strategy research typically happens in Python using vectorized approaches, while production trading systems are built separately using event-driven architectures in compiled languages.

NautilusTrader removes this separation.

A Rust-native core provides a deterministic event-driven runtime for both research and live execution, while Python serves as the control plane. The same architecture, execution semantics, and time model operate across both environments, allowing strategies to move from research to production without reimplementation.

Python bindings are provided via PyO3, with an ongoing migration from Cython. No Rust toolchain is required at install time.

Use cases

There are three main use cases for this software package:

- Backtest trading systems on historical data (`backtest`).
- Simulate trading systems with real-time data and virtual execution (`sandbox`).
- Deploy trading systems live on real or paper accounts (`live`).

The codebase provides a framework for building the software layer of systems that achieve the above. The default `backtest` and `live` system implementations live in their respectively named subpackages. A `sandbox` environment can be built using the sandbox adapter.



- All examples will use these default system implementations.
- We consider trading strategies to be subcomponents of end-to-end trading systems, these systems include the application and infrastructure layers.

Distributed

The platform integrates into larger distributed systems. Nearly all configuration and domain objects serialize using JSON, MessagePack, or Apache Arrow (Feather) for communication over the network.

Common core

The common system core is used by all node environment contexts (`backtest`, `sandbox`, and `live`). User-defined `Actor`, `Strategy` and `ExecAlgorithm` components are managed consistently across these environment contexts.

Backtesting

Feed data to a `BacktestEngine` either directly or through a higher-level `BacktestNode` and `ParquetDataCatalog`, then run the data through the system with nanosecond resolution.

Live trading

A `TradingNode` ingests data and events from multiple data and execution clients, supporting both demo/paper trading accounts and real accounts. Running asynchronously on a single event loop provides high performance, with the option to use the uvloop implementation (available for Linux and macOS) for additional throughput.

Domain model

The platform features a trading domain model that includes various value types such as `Price` and `Quantity`, as well as more complex entities such as `Order` and `Position` objects, which are used to aggregate multiple events to determine state.

Timestamps

All timestamps use nanosecond precision in UTC.

Timestamp strings follow ISO 8601 (RFC 3339) format with either 9 digits (nanoseconds) or 3 digits (milliseconds) of decimal precision, (but mostly nanoseconds) always maintaining all digits including trailing zeros. These can be seen in log messages, and debug/display outputs for objects.

A timestamp string consists of:

- Full date component always present: `YYYY-MM-DD`.
- `T` separator between date and time components.
- Always nanosecond precision (9 decimal places) or millisecond precision (3 decimal places) for certain cases such as GTD expiry times.
- Always UTC timezone designated by `Z` suffix.

Example: `2024-01-05T15:30:45.123456789Z`

For the complete specification, refer to [RFC 3339: Date and Time on the Internet](#).

UUIDs

The platform uses Universally Unique Identifiers (UUID) version 4 (RFC 4122) for unique identifiers. Our high-performance implementation uses the `uuid` crate for correctness validation when parsing from strings, ensuring input UUIDs comply with the specification.

A valid UUID v4 consists of:

- 32 hexadecimal digits displayed in 5 groups.
- Groups separated by hyphens: `8-4-4-4-12` format.
- Version 4 designation (indicated by the third group starting with "4").
- RFC 4122 variant designation (indicated by the fourth group starting with "8", "9", "a", or "b").

Example: `2d89666b-1a1e-4a75-b193-4eb3b454c757`

For the complete specification, refer to [RFC 4122: A Universally Unique Identifier \(UUID\) URN Namespace](#).

Data types

The following market data types can be requested historically, and also subscribed to as live streams when available from a venue / data provider, and implemented in an integrations adapter.

- `OrderBookDelta` (L1/L2/L3)
- `OrderBookDeltas` (container type)
- `OrderBookDepth10` (fixed depth of 10 levels per side)
- `QuoteTick`
- `TradeTick`
- `Bar`
- `Instrument`
- `InstrumentStatus`

- `InstrumentClose`

The following `PriceType` options can be used for bar aggregations:

- `BID`
- `ASK`
- `MID`
- `LAST`

Bar aggregations

The following `BarAggregation` methods are available:

- `MILLISECOND`
- `SECOND`
- `MINUTE`
- `HOURL`
- `DAY`
- `WEEK`
- `MONTH`
- `YEAR`
- `TICK`
- `VOLUME`
- `VALUE` (a.k.a Dollar bars)
- `RENKO` (price-based bricks)
- `TICK_IMBALANCE`
- `TICK_RUNS`
- `VOLUME_IMBALANCE`

- `VOLUME_RUNS`
- `VALUE_IMBALANCE`
- `VALUE_RUNS`

All listed aggregations are implemented for internal aggregation. Information-driven aggregations require `TradeTick` data.

The price types and bar aggregations can be combined with step sizes ≥ 1 in any way through a `BarSpecification`. This allows alternative bars to be aggregated for live trading.

Account types

The following account types are available for both live and backtest environments:

- `Cash` single-currency (base currency)
- `Cash` multi-currency
- `Margin` single-currency (base currency)
- `Margin` multi-currency
- `Betting` single-currency

Order types

The following order types are available (when possible on a venue):

- `MARKET`
- `LIMIT`
- `STOP_MARKET`
- `STOP_LIMIT`
- `MARKET_TO_LIMIT`
- `MARKET_IF_TOUCHED`

- `LIMIT_IF_TOUCHED`
- `TRAILING_STOP_MARKET`
- `TRAILING_STOP_LIMIT`

Value types

The following value types are backed by either 128-bit or 64-bit raw integer values, depending on the precision mode used during compilation.

- `Price`
- `Quantity`
- `Money`

High-precision mode (128-bit)

When the `high-precision` feature flag is enabled (default), values use the specification:

Type	Raw backing	Max precision	Min value	Max value
<code>Price</code>	<code>i128</code>	16	-17,014,118,346,046	17,014,118,346,046
<code>Money</code>	<code>i128</code>	16	-17,014,118,346,046	17,014,118,346,046
<code>Quantity</code>	<code>u128</code>	16	0	34,028,236,692,093

Standard-precision mode (64-bit)

When the `high-precision` feature flag is disabled, values use the specification:

Type	Raw backing	Max precision	Min value	Max value
<code>Price</code>	<code>i64</code>	9	-9,223,372,036	9,223,372,036
<code>Money</code>	<code>i64</code>	9	-9,223,372,036	9,223,372,036

Type	Raw backing	Max precision	Min value	Max value
Quantity	u64	9	0	18,446,744,073

< Concepts

Previous Page

Architecture >

Next Page